READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))


all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")
```

```
print(f"pneumon : {len(all_pneumonia_files)}")
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

```
train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

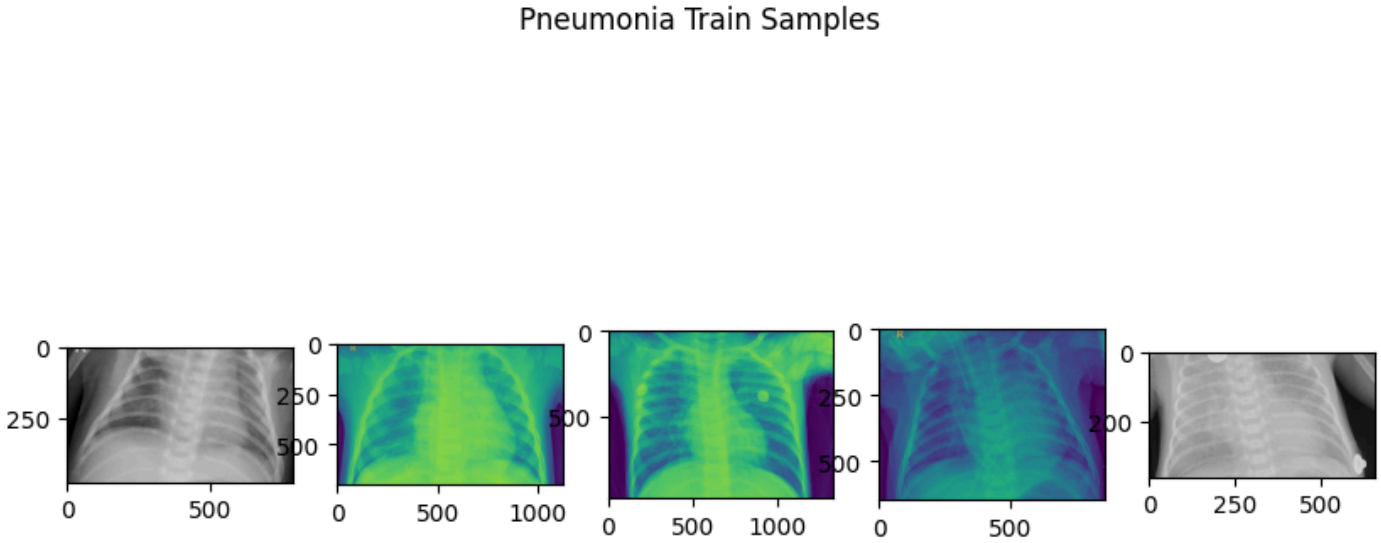
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

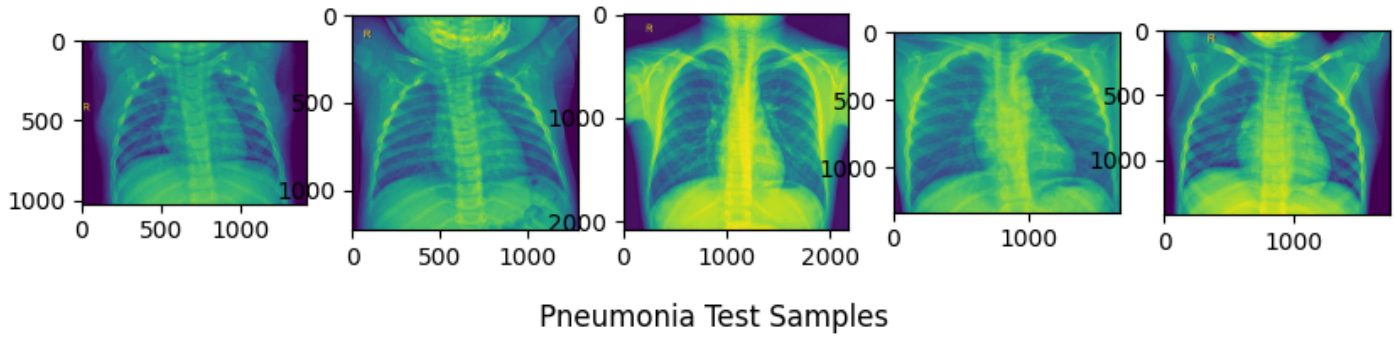
#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]
```

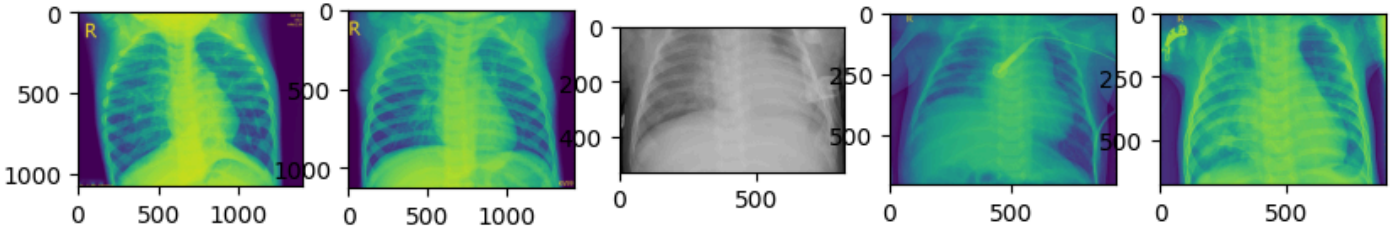
```
from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

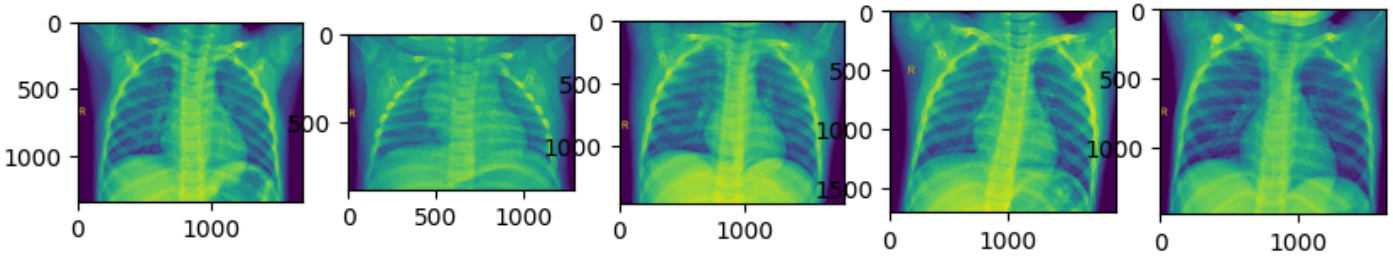
show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```


Pneumonia Train Samples
Normal Train Samples


Pneumonia Test Samples



Normal Test Samples



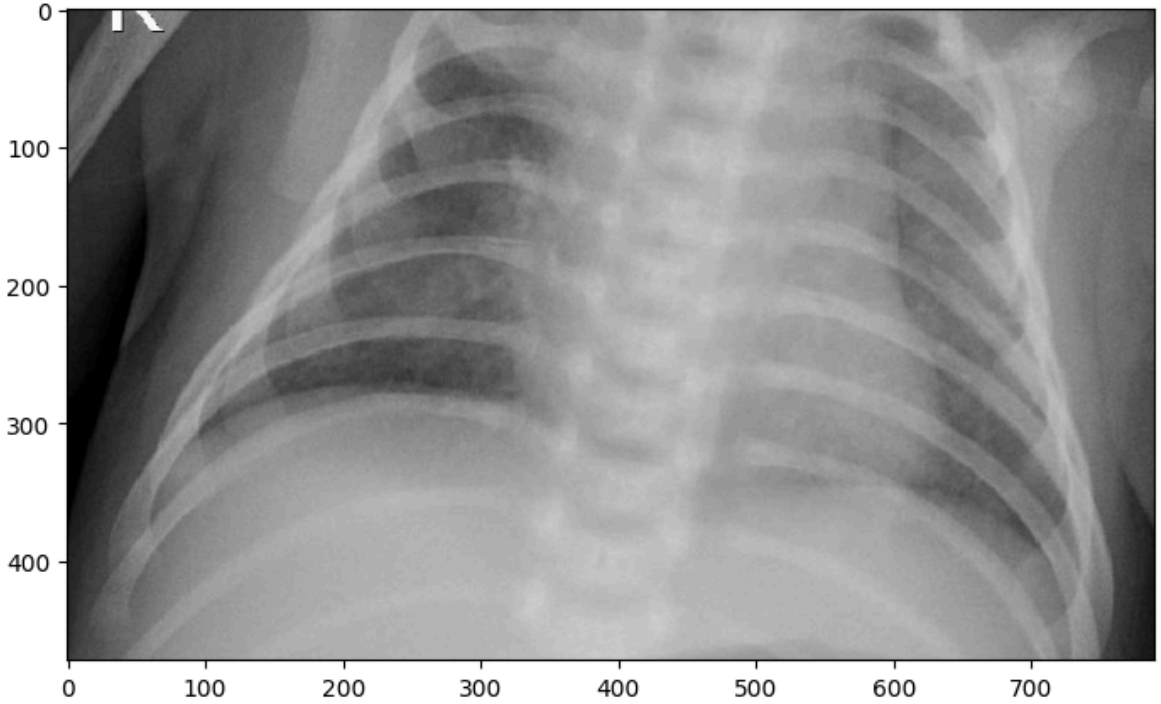
RESIZE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

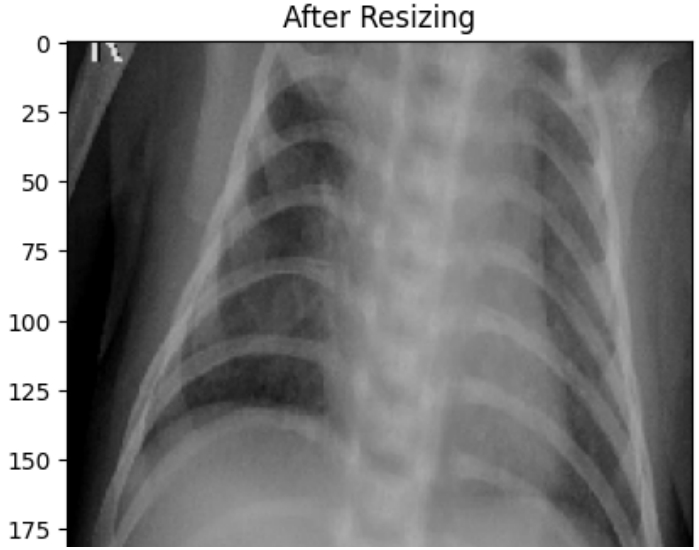
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

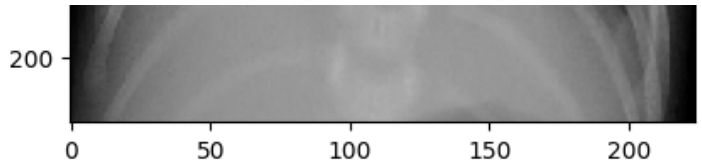
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)


show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```




```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.1176
Text(0.5, 1.0, 'After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features



import cv2
```

```
def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(392.6822788950354),
 'contrast_std': np.float64(186.83649114967167),
 'contrast_var': np.float64(34907.87442512134),
 'dissimilarity_mean': np.float64(7.149433041375201),
 'dissimilarity_std': np.float64(2.248002085050434),
 'dissimilarity_var': np.float64(5.053513374391098),
 'homogeneity_mean': np.float64(0.21032457077217873),
 'homogeneity_std': np.float64(0.044473886157315894),
 'homogeneity_var': np.float64(0.0019779265499338945),
 'energy_mean': np.float64(0.021551966679647067),
 'energy_std': np.float64(0.0025198063646927333),
 'energy_var': np.float64(6.349424115546009e-06),
 'correlation_mean': np.float64(0.8878819130695859),
 'correlation_std': np.float64(0.05498269265774188),
 'correlation_var': np.float64(0.003023096491895703),
 'ASM_mean': np.float64(0.0004708366918761635),
 'ASM_std': np.float64(0.0001107862326594272),
 'ASM_var': np.float64(1.2273589346868734e-08),
 'entropy': np.float64(142.80640495208078)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
```

```
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)
```

```
# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.3,
    callbacks=callbacks,
```

```
        class_weight=class_weights,
    )
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	2,560
batch_normalization_2 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
batch_normalization_3 (BatchNormalization)	(None, 64)	256
dropout_3 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

```
 Total params: 11,649 (45.50 KB)
 Trainable params: 11,265 (44.00 KB)
 Non-trainable params: 384 (1.50 KB)
Epoch 1/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.6969 - auc_1: 0.7796 - loss: 0.6566 - val_accuracy: 0.7630 - val_auc_1: 0.0000e+00 - val_loss: 0.61
Epoch 2/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8469 - auc_1: 0.9255 - loss: 0.3844 - val_accuracy: 0.7856 - val_auc_1: 0.0000e+00 - val_loss: 0.580
Epoch 3/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8721 - auc_1: 0.9366 - loss: 0.3506 - val_accuracy: 0.7560 - val_auc_1: 0.0000e+00 - val_loss: 0.587
Epoch 4/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8685 - auc_1: 0.9245 - loss: 0.3646 - val_accuracy: 0.8251 - val_auc_1: 0.0000e+00 - val_loss: 0.511
Epoch 5/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8643 - auc_1: 0.9330 - loss: 0.3480 - val_accuracy: 0.8491 - val_auc_1: 0.0000e+00 - val_loss: 0.463
Epoch 6/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8670 - auc_1: 0.9364 - loss: 0.3386 - val_accuracy: 0.8420 - val_auc_1: 0.0000e+00 - val_loss: 0.454
Epoch 7/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8829 - auc_1: 0.9495 - loss: 0.2976 - val_accuracy: 0.8660 - val_auc_1: 0.0000e+00 - val_loss: 0.394
Epoch 8/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8750 - auc_1: 0.9361 - loss: 0.3362 - val_accuracy: 0.8717 - val_auc_1: 0.0000e+00 - val_loss: 0.402
Epoch 9/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8854 - auc_1: 0.9406 - loss: 0.3266 - val_accuracy: 0.8731 - val_auc_1: 0.0000e+00 - val_loss: 0.381
Epoch 10/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8746 - auc_1: 0.9481 - loss: 0.2984 - val_accuracy: 0.8914 - val_auc_1: 0.0000e+00 - val_loss: 0.376
Epoch 11/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8942 - auc_1: 0.9526 - loss: 0.2892 - val_accuracy: 0.8914 - val_auc_1: 0.0000e+00 - val_loss: 0.35
Epoch 12/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9048 - auc_1: 0.9590 - loss: 0.2632 - val_accuracy: 0.8858 - val_auc_1: 0.0000e+00 - val_loss: 0.35
```

```
Epoch 13/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9007 - auc_1: 0.9526 - loss: 0.2862 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.365
Epoch 14/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8996 - auc_1: 0.9551 - loss: 0.2802 - val_accuracy: 0.8999 - val_auc_1: 0.0000e+00 - val_loss: 0.33
Epoch 15/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8914 - auc_1: 0.9527 - loss: 0.2838 - val_accuracy: 0.8999 - val_auc_1: 0.0000e+00 - val_loss: 0.342
Epoch 16/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9042 - auc_1: 0.9522 - loss: 0.2827 - val_accuracy: 0.8942 - val_auc_1: 0.0000e+00 - val_loss: 0.36
Epoch 17/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8870 - auc_1: 0.9484 - loss: 0.2946 - val_accuracy: 0.8886 - val_auc_1: 0.0000e+00 - val_loss: 0.36
Epoch 18/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9091 - auc_1: 0.9628 - loss: 0.2515 - val_accuracy: 0.8900 - val_auc_1: 0.0000e+00 - val_loss: 0.376
Epoch 19/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8769 - auc_1: 0.9441 - loss: 0.3139 - val_accuracy: 0.8956 - val_auc_1: 0.0000e+00 - val_loss: 0.355
Epoch 20/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8865 - auc_1: 0.9617 - loss: 0.2607 - val_accuracy: 0.9013 - val_auc_1: 0.0000e+00 - val_loss: 0.358
Epoch 21/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9107 - auc_1: 0.9589 - loss: 0.2711 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.399
Epoch 22/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9250 - auc_1: 0.9700 - loss: 0.2269 - val_accuracy: 0.8942 - val_auc_1: 0.0000e+00 - val_loss: 0.358
Epoch 23/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9018 - auc_1: 0.9649 - loss: 0.2501 - val_accuracy: 0.8801 - val_auc_1: 0.0000e+00 - val_loss: 0.395
Epoch 24/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8948 - auc_1: 0.9569 - loss: 0.2663 - val_accuracy: 0.8999 - val_auc_1: 0.0000e+00 - val_loss: 0.347
Epoch 25/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8925 - auc_1: 0.9564 - loss: 0.2733 - val_accuracy: 0.8759 - val_auc_1: 0.0000e+00 - val_loss: 0.414
Epoch 26/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8934 - auc_1: 0.9555 - loss: 0.2876 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.403
Epoch 27/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9038 - auc_1: 0.9598 - loss: 0.2608 - val_accuracy: 0.8745 - val_auc_1: 0.0000e+00 - val_loss: 0.436
Epoch 28/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9101 - auc_1: 0.9676 - loss: 0.2317 - val_accuracy: 0.8815 - val_auc_1: 0.0000e+00 - val_loss: 0.411
Epoch 29/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9135 - auc_1: 0.9651 - loss: 0.2418 - val_accuracy: 0.8815 - val_auc_1: 0.0000e+00 - val_loss: 0.398
Epoch 30/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9044 - auc_1: 0.9604 - loss: 0.2553 - val_accuracy: 0.9140 - val_auc_1: 0.0000e+00 - val_loss: 0.349
Epoch 31/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9053 - auc_1: 0.9586 - loss: 0.2610 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.396
Epoch 32/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8995 - auc_1: 0.9576 - loss: 0.2622 - val_accuracy: 0.8815 - val_auc_1: 0.0000e+00 - val_loss: 0.424
Epoch 33/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9191 - auc_1: 0.9664 - loss: 0.2391 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.387
Epoch 34/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8986 - auc_1: 0.9564 - loss: 0.2672 - val_accuracy: 0.8900 - val_auc_1: 0.0000e+00 - val_loss: 0.380
Epoch 35/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9014 - auc_1: 0.9600 - loss: 0.2555 - val_accuracy: 0.8829 - val_auc_1: 0.0000e+00 - val_loss: 0.409
Epoch 36/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9142 - auc_1: 0.9705 - loss: 0.2240 - val_accuracy: 0.8928 - val_auc_1: 0.0000e+00 - val_loss: 0.388
Epoch 37/100
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8875 - auc_1: 0.9559 - loss: 0.2692 - val_accuracy: 0.9027 - val_auc_1: 0.0000e+00 - val_loss: 0.377
Epoch 38/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8995 - auc_1: 0.9667 - loss: 0.2344 - val_accuracy: 0.8942 - val_auc_1: 0.0000e+00 - val_loss: 0.381
Epoch 39/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9069 - auc_1: 0.9648 - loss: 0.2428 - val_accuracy: 0.8773 - val_auc_1: 0.0000e+00 - val_loss: 0.406
Epoch 40/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9182 - auc_1: 0.9659 - loss: 0.2328 - val_accuracy: 0.8787 - val_auc_1: 0.0000e+00 - val_loss: 0.408
Epoch 41/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9002 - auc_1: 0.9665 - loss: 0.2355 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.39
Epoch 42/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9086 - auc_1: 0.9626 - loss: 0.2485 - val_accuracy: 0.8858 - val_auc_1: 0.0000e+00 - val_loss: 0.398
Epoch 43/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9224 - auc_1: 0.9686 - loss: 0.2243 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.39
Epoch 44/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8939 - auc_1: 0.9605 - loss: 0.2519 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.394
Epoch 45/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9070 - auc_1: 0.9621 - loss: 0.2454 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.40
Epoch 46/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9002 - auc_1: 0.9624 - loss: 0.2433 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.397
Epoch 47/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8962 - auc_1: 0.9688 - loss: 0.2342 - val_accuracy: 0.8843 - val_auc_1: 0.0000e+00 - val_loss: 0.407
Epoch 48/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9327 - auc_1: 0.9750 - loss: 0.2003 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.409
Epoch 49/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9135 - auc_1: 0.9713 - loss: 0.2227 - val_accuracy: 0.8858 - val_auc_1: 0.0000e+00 - val_loss: 0.411
Epoch 50/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9279 - auc_1: 0.9742 - loss: 0.2109 - val_accuracy: 0.8900 - val_auc_1: 0.0000e+00 - val_loss: 0.403
Epoch 51/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9135 - auc_1: 0.9673 - loss: 0.2319 - val_accuracy: 0.8773 - val_auc_1: 0.0000e+00 - val_loss: 0.423
Epoch 52/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9182 - auc_1: 0.9681 - loss: 0.2284 - val_accuracy: 0.8759 - val_auc_1: 0.0000e+00 - val_loss: 0.416
Epoch 53/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9169 - auc_1: 0.9717 - loss: 0.2153 - val_accuracy: 0.8801 - val_auc_1: 0.0000e+00 - val_loss: 0.415
Epoch 54/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9052 - auc_1: 0.9666 - loss: 0.2373 - val_accuracy: 0.8872 - val_auc_1: 0.0000e+00 - val_loss: 0.390
Epoch 55/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9196 - auc_1: 0.9714 - loss: 0.2207 - val_accuracy: 0.8801 - val_auc_1: 0.0000e+00 - val_loss: 0.408
Epoch 56/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9186 - auc_1: 0.9674 - loss: 0.2377 - val_accuracy: 0.8829 - val_auc_1: 0.0000e+00 - val_loss: 0.392
Epoch 57/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9138 - auc_1: 0.9675 - loss: 0.2272 - val_accuracy: 0.8759 - val_auc_1: 0.0000e+00 - val_loss: 0.417
Epoch 58/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9044 - auc_1: 0.9635 - loss: 0.2485 - val_accuracy: 0.8858 - val_auc_1: 0.0000e+00 - val_loss: 0.396
Epoch 59/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9206 - auc_1: 0.9729 - loss: 0.2182 - val_accuracy: 0.8773 - val_auc_1: 0.0000e+00 - val_loss: 0.408
Epoch 60/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9179 - auc_1: 0.9671 - loss: 0.2296 - val_accuracy: 0.8787 - val_auc_1: 0.0000e+00 - val_loss: 0.418
Epoch 61/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9338 - auc_1: 0.9803 - loss: 0.1782 - val_accuracy: 0.8773 - val_auc_1: 0.0000e+00 - val_loss: 0.424
```

```
Epoch 62/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9087 - auc_1: 0.9675 - loss: 0.2410 - val_accuracy: 0.8773 - val_auc_1: 0.0000e+00 - val_loss: 0.417
Epoch 63/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9263 - auc_1: 0.9699 - loss: 0.2211 - val_accuracy: 0.8787 - val_auc_1: 0.0000e+00 - val_loss: 0.415
Epoch 64/100
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9069 - auc_1: 0.9715 - loss: 0.2231 - val_accuracy: 0.8815 - val_auc_1: 0.0000e+00 - val_loss: 0.420
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
- Loss: 0.3291
- AUC: 0.9388
- Accuracy: 0.8807
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.88      0.88      0.88       394
           1       0.88      0.88      0.88       394

    accuracy                           0.88       788
   macro avg       0.88      0.88      0.88       788
weighted avg       0.88      0.88      0.88       788
```

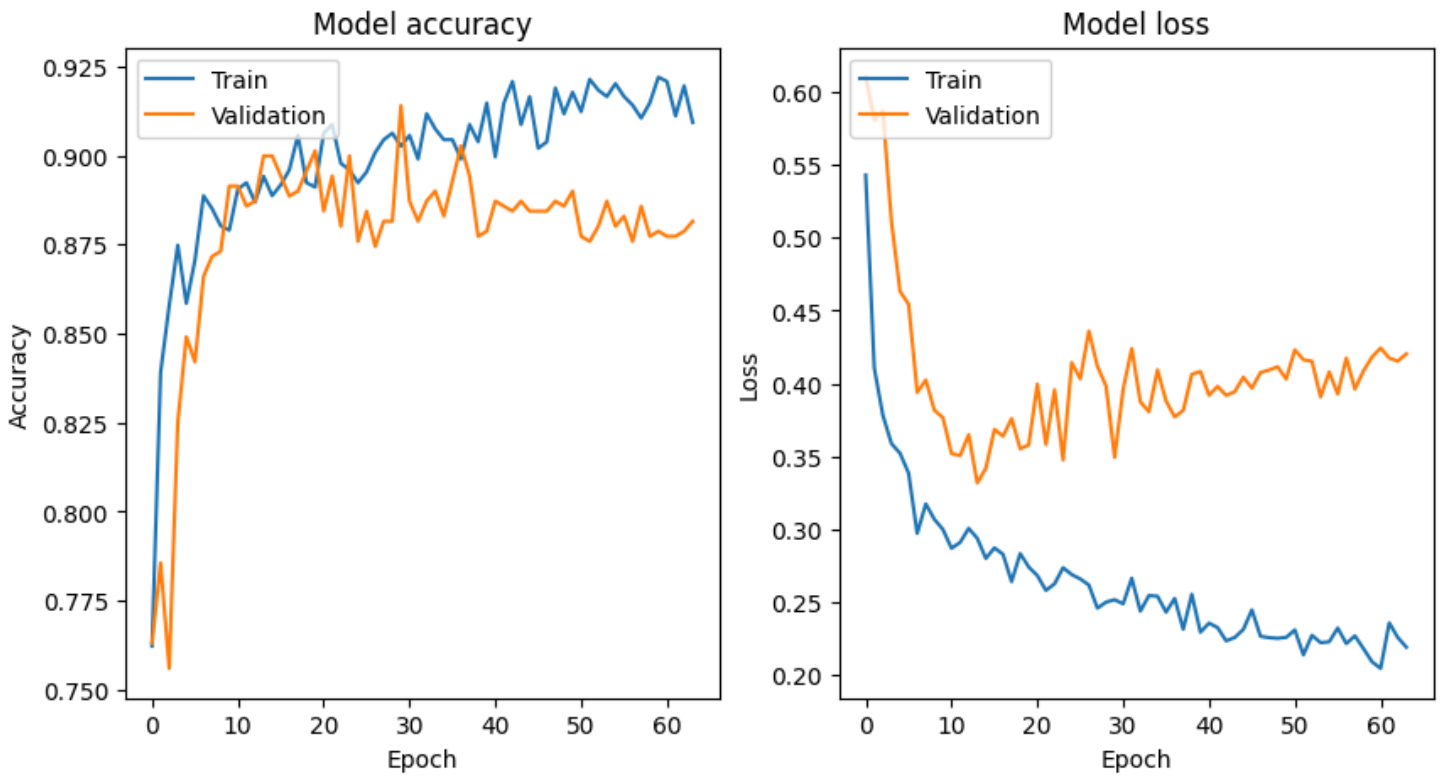
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

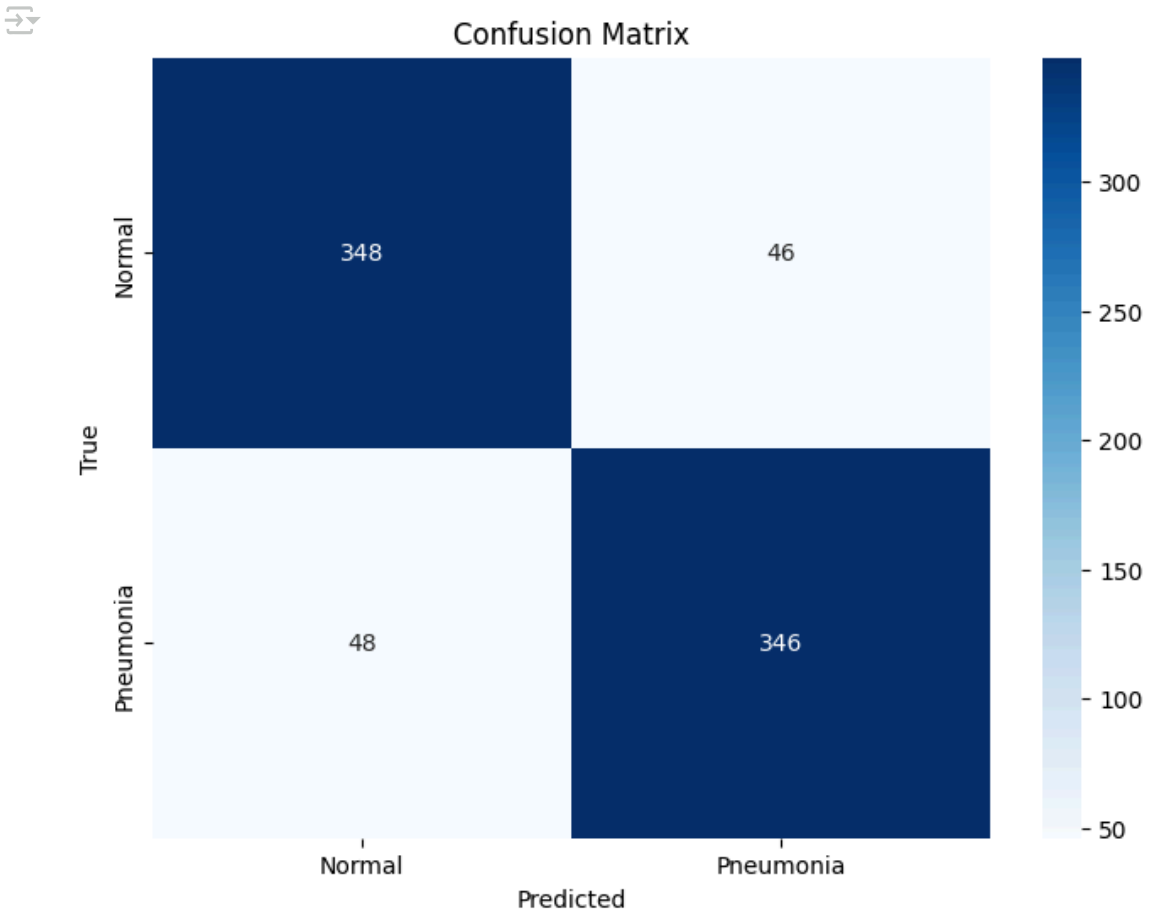
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
```

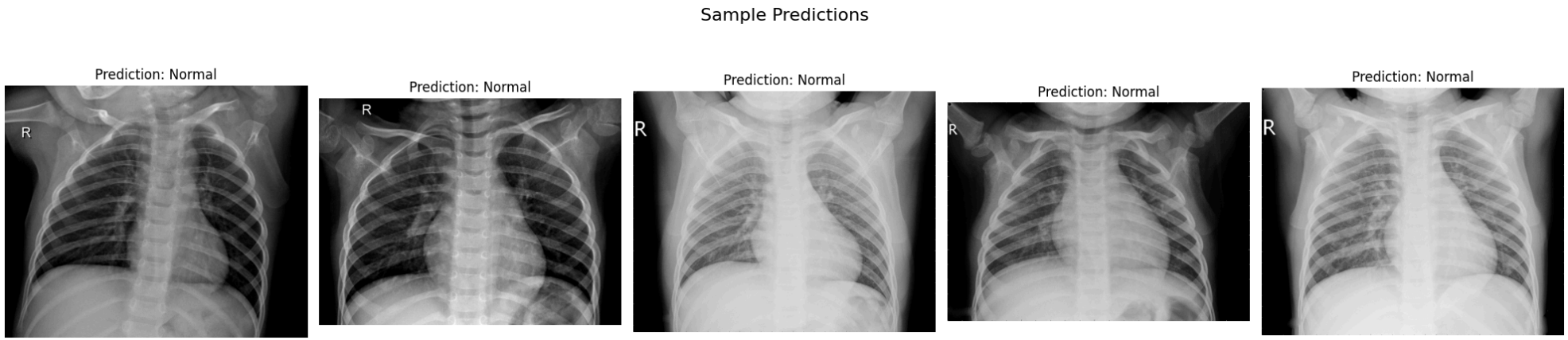
```
            axes[i].set_title(f"Prediction: {prediction_for_sample}")
            axes[i].axis('off')

        except Exception as e:
            print(f"Error processing {image_file}: {e}")

    plt.tight_layout()
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
7	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal




```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
```

```
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

    except Exception as e:
```

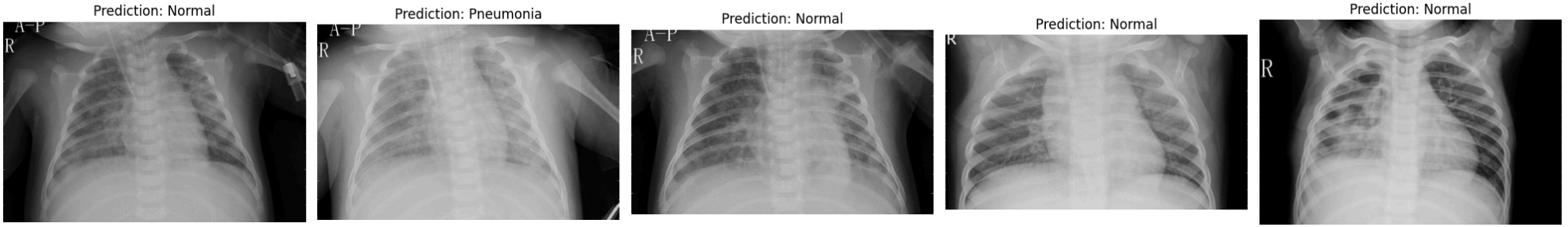
```
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Normal
7	/content/drive/MyDrive/TA/Dataset/val/PNEUMONI...	Pneumonia

Sample Predictions




```
# prompt: uji data glcm ini juga dengan svm

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8642

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.88      0.87       394
           1       0.88      0.85      0.86       394

    accuracy                           0.86       788
   macro avg       0.86      0.86      0.86       788
weighted avg       0.86      0.86      0.86       788
```

